

STRATASEARCH

INTELLIGENT DOCUMENT SEARCH & ANALYSIS PLATFORM

DSA-III Project – Review 3 | Group 7 (A-7_DSA-3_G-7)

CORE PROBLEM STATEMENT

Searching large technical document collections efficiently is inherently an algorithm-selection and performance-analysis problem.

// PROJECT TEAM

RITHVIK

Core Algorithms

SASHANK

Backend Architecture

ARNAVI

Frontend Interface

// SYSTEM VISUAL PIPELINE FLOW

STEP 01
DOCUMENTS

Large Technical Corpora



STEP 02
SEARCH QUERY

User Search & Params



STEP 03 [CORE]
ALGORITHM SELECTION
Dynamic DSA Optimization



STEP 04
RESULTS
Analysis & Metrics

PROJECT GUIDE : Dr. AFTAB YASEEN

One Search Strategy Does Not Fit Every Workload

Search Engine Limits

Traditional keyword search engines hide algorithmic decisions under generic indexes.

Different query workloads demand distinct underlying data structures and strategies to optimize performance and resource usage.

OBJECTIVE WORKFLOW

1. Analyze specific query workload
2. Select optimal candidate algorithm
3. Explain architectural trade-offs
4. Benchmark and measure results

Workload-To-Algorithm Mapping

WORKLOAD: SINGLE EXACT PATTERN

KMP, Z-Algorithm, Rabin-Karp

WORKLOAD: MULTIPLE PATTERNS SIMULTANEOUSLY

Aho-Corasick Automaton

WORKLOAD: REPEATED SEARCH ON SAME CORPUS

Suffix Array + LCP Array

FROM STATIC ALGORITHMS TO WORKLOAD-AWARE SEARCH

INNOVATION: Dynamically selecting and explaining classical DSA algorithms based on workload characteristics.

WORKLOAD-AWARE EXECUTION PIPELINE



ALGORITHM POOL

CLASSICAL DSA ALGORITHMS

- **KMP (Knuth-Morris-Pratt):** Optimal linear search with prefix function avoiding backtracking.
- **Rabin-Karp:** Rolling hash matching for multi-pattern and exact queries.
- **Z Algorithm:** Fast exact pattern matching using Z-array linear processing.
- **Aho-Corasick:** Automaton-driven dictionary search across massive corpora.
- **Suffix Array + LCP:** Substring searching with dynamic LCP array indexing.

KEY HIGHLIGHTS

SYSTEM NOVELTY & ARCHITECTURE

Transparent CostModel

Evaluates workload characteristics to calculate accurate execution cost vectors.

Explainable Recommendations

Provides step-by-step logic and dynamic explanations for algorithm selection.

Measurable Execution

Captures real-time telemetry to validate performance metrics and refine ranking.

Adapting Algorithm Selection to Specific Workloads

The transparent heuristic planner dynamically adapts to query types:

Single Pattern

("machine learning")

Selected Algorithms:

- KMP
- Z Algorithm
- Rabin-Karp

Multiple Patterns

("AI", "NLP")

Selected Algorithm:

- Aho-Corasick

Repeated Queries

Selected Index Strategy:

- Suffix Array + LCP

(amortized index build cost)

KEY TAKEAWAY: Adapts to workload rather than forcing a single algorithm.

Five Core Algorithms and Theoretical Complexities

Algorithm	Primary Data Structure	Primary Use Case	Time Complexity
KMP	LPS Array	Single-pattern	$O(N+M)$
Rabin-Karp	Rolling Hash	Pattern Matching	Avg $O(N+M)$
Z Algorithm	Z-Array	Linear Preprocessing	$O(N+M)$
Aho-Corasick	Trie + Failure Links	Multi-pattern	$O(N + \text{pattern len} + \text{matches})$
Suffix Array + LCP	Suffix Array & LCP Array	Indexed Repeated Search	Build $O(N \log N)$

Empirical Measurement and Benchmarking Framework

EVALUATION SETUP

Same corpus + Same workload + Multiple algorithms =
Comparable measurement.

CAPTURED TELEMETRY

1.2ms

Execution
Time

14.2k

Comparison
Count

1,024

Match Count

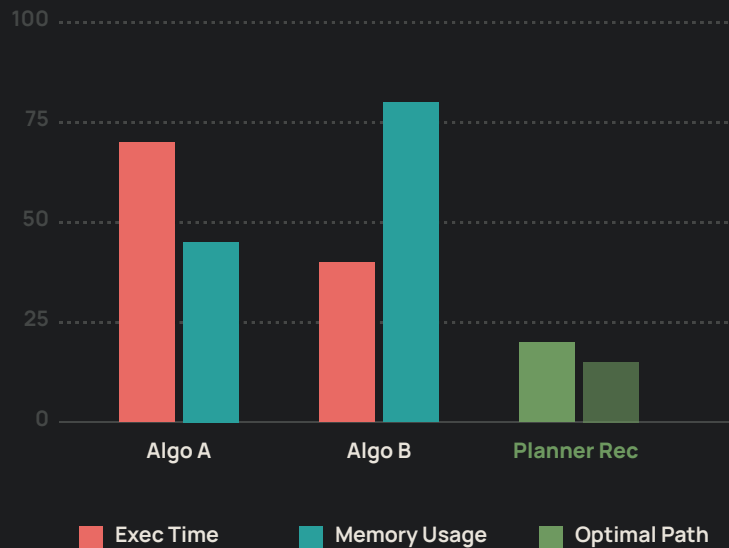
64MB

Memory
Metrics

VALIDATION

Benchmarking directly validates whether the planner's algorithm recommendation is optimal for the observed workload.

METHODOLOGY EVALUATION BENCHMARK



Comparative algorithm telemetry confirms Planner Recommendation
minimizes overhead across the corpus.

INTERACTIVE VISUAL LEARNING WORKBENCH

Exposes algorithm behavior rather than hiding it inside a black-box API.

FRONTEND UI WORKBENCH • DISSEMINATION & ACCESSIBILITY

Document Upload & Query Panel

Upload source documents and input complex algorithmic evaluation queries.

Planner Explanation & Visual Playback

Inspect visual planner breakdowns with full step-by-step state inspection.

Playback Step Controls

▶ PLAY

⏸ PAUSE

⏭ STEP

⚡ SPEED: 1.5x

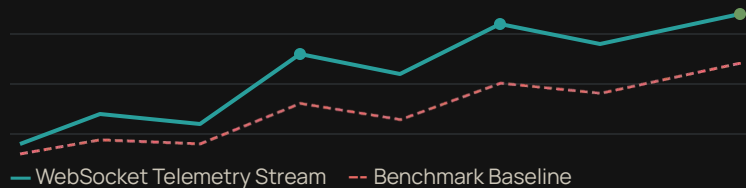
TECH STACK ARCHITECTURE

- Backend: Spring Boot Engine
- Frontend: React + TypeScript + Vite
- Streaming: WebSocket progress streaming

LIVE WEBSOCKET TELEMETRY & BENCHMARK

[WS_CONNECT] ws://backend/api/v1/telemetry

- > Step 01: Document ingestion & chunk indexing... DONE
- > Step 02: Planner path generated [Graph Depth: 4]
- > Step 03: Streaming visual execution telemetry...
- > Benchmark: Execution latency = 14ms (Real-time)



BENCHMARK WORKSPACE CAPABILITY

Runs side-by-side performance profiling across document query strategies.

CURRENT STATUS AND FINAL MILESTONE ROADMAP

REVIEW 3 (75% - COMPLETE)

Completed Deliverables:

- ✓ Core DSA engine
- ✓ Query planner & cost model
- ✓ Benchmarking & telemetry
- ✓ Interactive workbench

REVIEW 4 (25% - PLANNED)

Upcoming Tasks:

- Deeper corpus profiling
- Levenshtein Edit Distance & LCS
- Document similarity & duplicate detection
- Containerization

FLOW: SEARCH → ANALYZE → CHOOSE → EXECUTE → EXPLAIN → EVALUATE